

Настройка рабочего места разработчика C++ в Windows (Clang 15)

Шаг 1: Удаление предыдущих версий ПО

Пропустите этот шаг, если вы настраиваете рабочее место в первый раз.

1. Удалите ранее установленное ПО:

- Qt Creator
- TensorCppDev - вместе с ним автоматически удаляются CMake, Conan и Build Tools.
- MinGW Development Toolkit
- библиотеки Boost - если их несколько и они установлены в разные директории, то удалите все.

2. Удалите директории C:\conan и C:\Users\[username]\conan, если они существуют.

3. Очистите переменную окружения Path от conan, boost, cmake, ninja.

4. Удалите все упоминания Qt Creator из директорий C:\Users\[username]\Appdata\....

5. Удалите сборочные файлы в проектах (CMakeLists.txt.user и всю сборочную директорию).

Шаг 2: Первичная настройка рабочего места

См. [руководство](#).

Шаг 3: Установка TensorCppDev

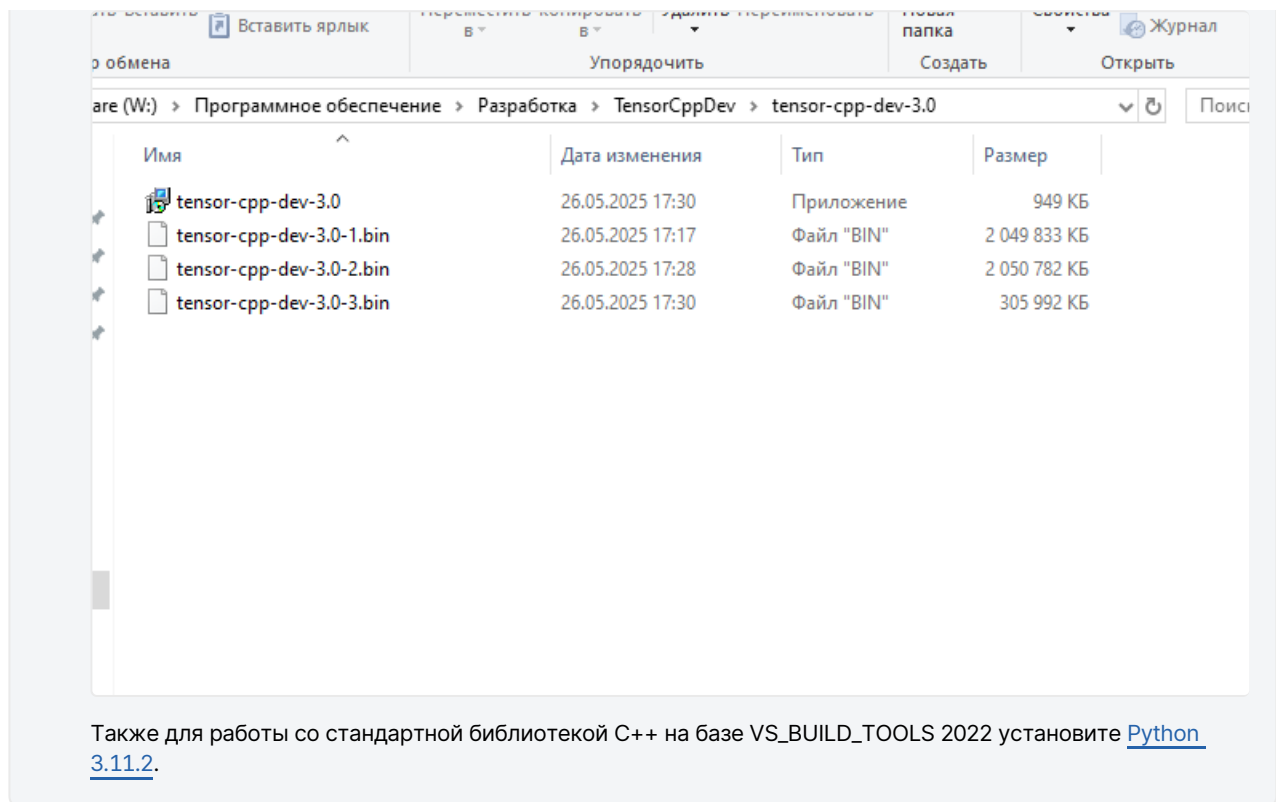
1. Скачайте **последнюю версию** инсталлятора TensorCppDev 2.x. На сетевом диске найдите директорию с инсталлятором последней версии, например, tensor-cpp-dev-2.2, и скопируйте ее на локальный диск.

```
1 | \\fs2.corp.tensor.ru\publicsoftware\Программное  
  | обеспечение\Разработка\TensorCppDev\tensor-cpp-dev-2.x\
```

2. Запустите установку TensorCppDev с помощью *.exe-файла внутри директории на локальном диске.

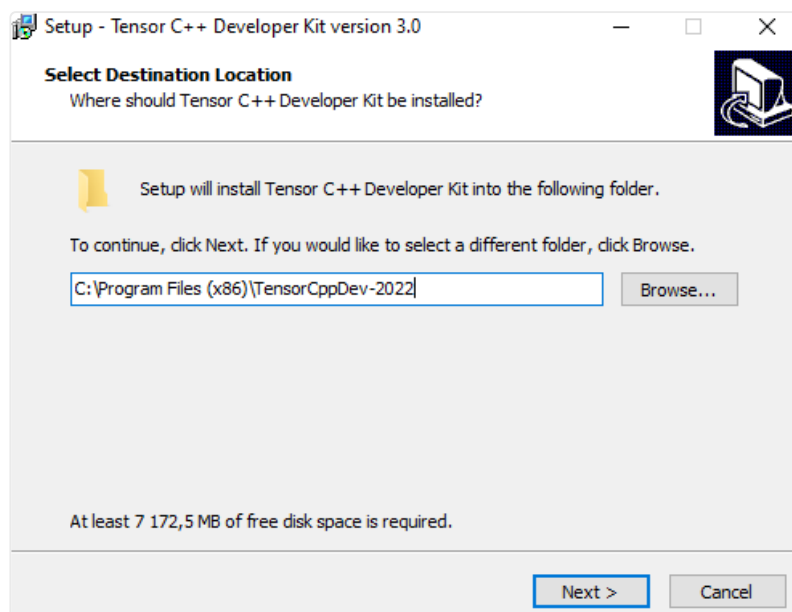


Для успешной установки необходимо около 20 Гб свободного места на диске C: для установки Visual Studio Build Tools 2022. Состав пакета установки tensor-cpp-dev-2.2:

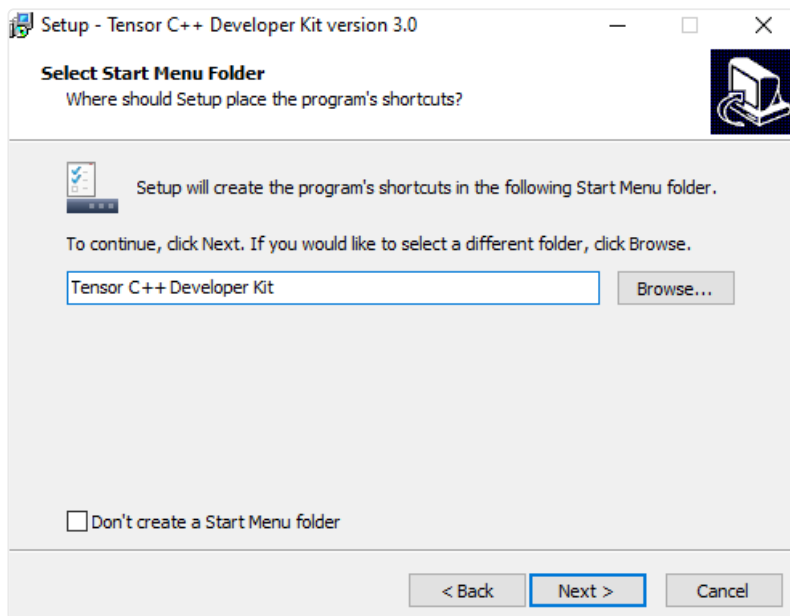


3. В процессе установки запустятся инсталляторы Visual Studio Build Tools 2022 (в его окне не нужно ничего нажимать, все произойдет автоматически) и в выбранную директорию добавятся несколько скриптов, упрощающих работу с тулчейном.

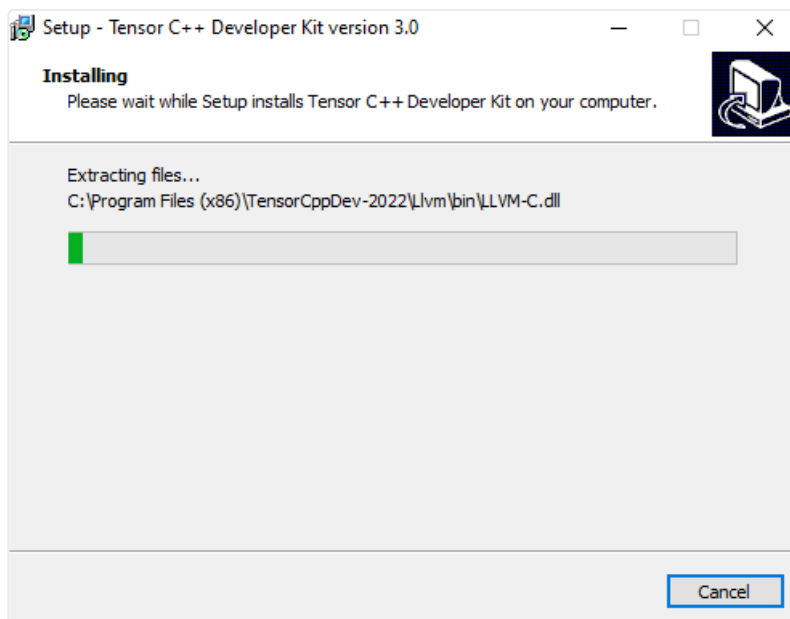
- Выбор директории, в которую установятся вспомогательные скрипты. Рекомендуется оставить по умолчанию.



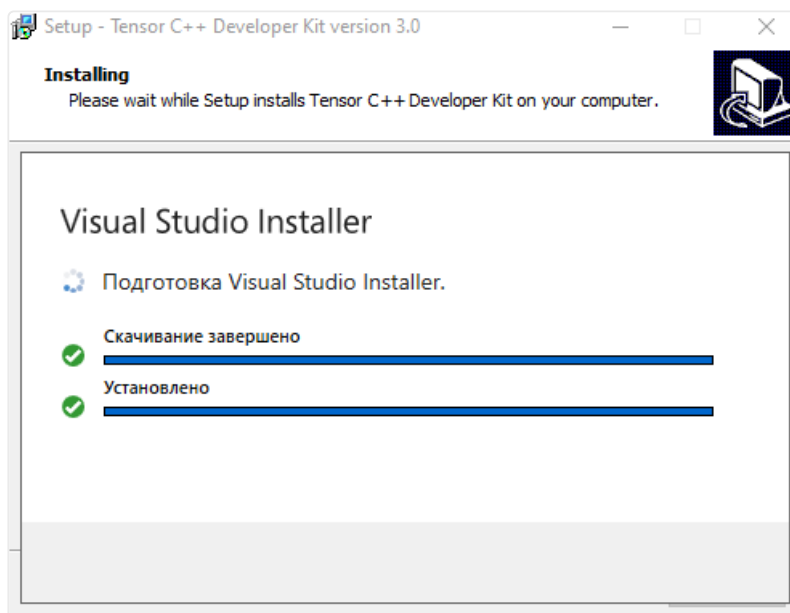
- Выбор папки в меню Пуск для создания ярлыков tensor-cpp-dev. Рекомендуется оставить по умолчанию.



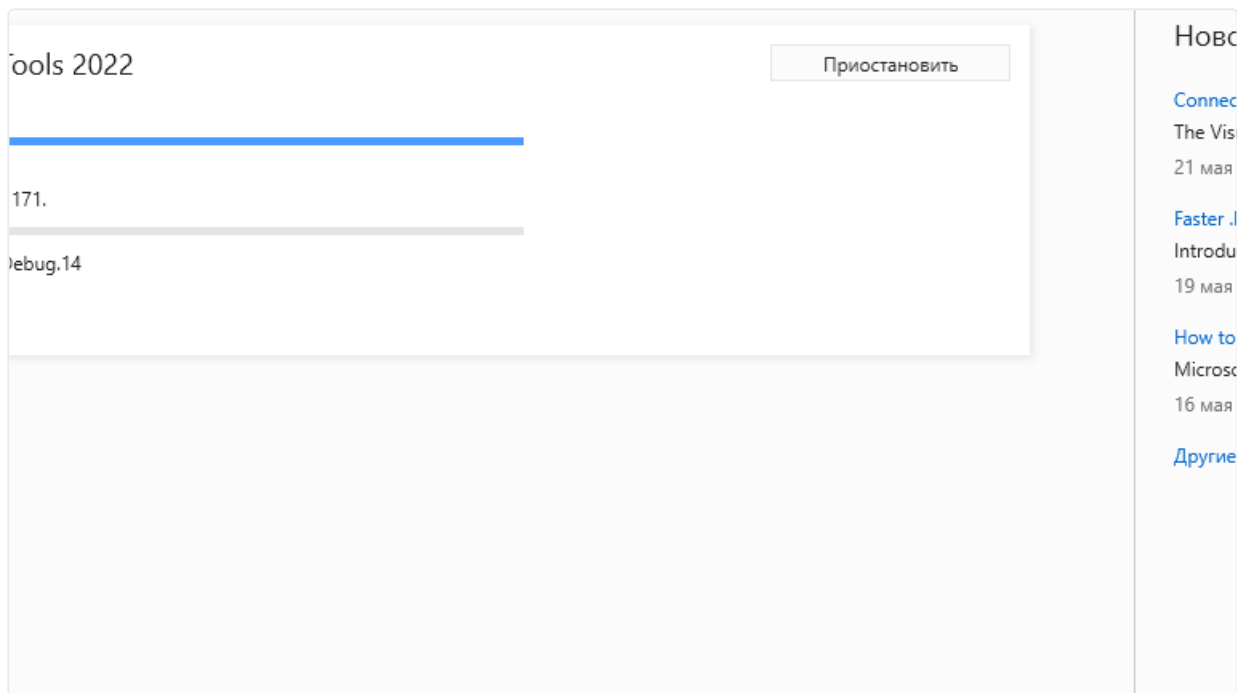
- Распаковка файлов инсталлятора, нужно немного подождать.



- Запуск инсталлятора Visual Studio Build Tools.



- В окне инсталлятора Visual Studio Build Tools ничего нажимать не нужно, все будет выполнено автоматически.



После завершения установки перезагрузите компьютер.



Не обновляйте Visual Studio Build Tools 2022!

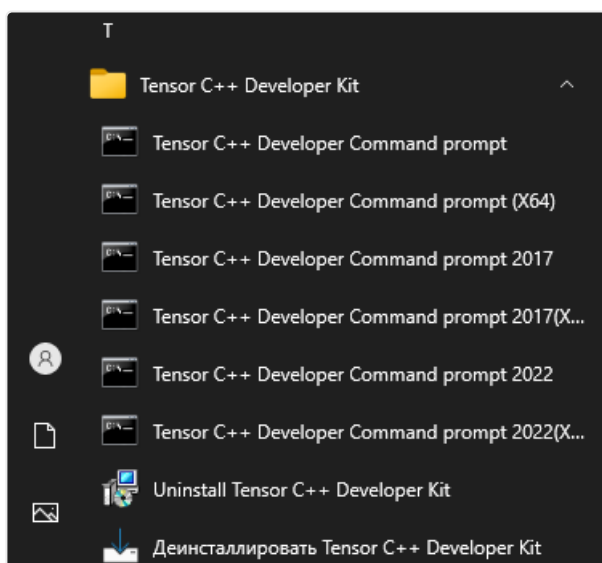
Visual Studio Build Tools 2022, установленный с помощью tensor-cpp-dev-2.x, должен быть версии 17.4.3.

Шаг 4: Настройка Conan

В настоящее время Conan входит в состав [SBIS SDK](#) и автоматически настраивается при запуске [CMake](#).

Шаг 5: Сборка через консоль

Для сборки через консоль в tensor-cpp-dev входят несколько скриптов, упрощающих настройку сборочного окружения. Найти их можно в папке с установленным TensorCppDev 2022 (по умолчанию C:\Program Files (x86)\TensorCppDev-2022), либо запустить консоль с нужным окружением из меню "Пуск".



Основной архитектурой приложений является Win32

Шаг 5.1: Сборка приложений Win32 через консоль

Для сборки 32-битного приложения запустите Tensor C++ Developer Command prompt 2022. 32-битные приложения в новом тулчейне собираются в режиме кросс-компиляции 64-битным компилятором, поэтому необходимо при запуске CMake передать (например, для TensorCppDev-2.2):

```
1 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-i686.cmake"
```

Команды для сборки простейшего приложения будут выглядеть так:

```
1 # конфигурация
2 cmake <path_to_my_source> -G Ninja -
  DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-i686.cmake"
3 # сборка
4 ninja
```

Шаг 5.2: Сборка приложений Win64 через консоль



Важно!

Для приложений Win64 платформа поддерживает только статическую конфигурацию. Установите дополнительно [Python 3.11 64 bit](#).

Для сборки 64-битного приложения запустите Tensor C++ Developer Command prompt 2022 (x64).

Команды для сборки простого приложения:

```
1 # конфигурация
2 cmake <path_to_my_source> -G Ninja -DSBIS_ARCHITECTURE=x86_64 -
  DSBIS_STATIC_BUILD=ON -DSBIS_STATIC_RUNTIME=ON
3 # сборка
4 ninja
```

Дополнительно для статической сборки вы можете использовать следующие аргументы:

```
1 -DSBIS_STATIC_BUILD=ON
2 -DSBIS_STATIC_RUNTIME=ON
```

Шаг 6: Настройка комплекта в IDE Qt Creator

Данная инструкция написана для Qt Creator 6.0.2. Установка подробно описана [здесь](#).

Перед настройкой комплекта проверьте, что у вас установлен CMake:

- Перейдите в меню **Инструменты (Tools)** → **Параметры... (Options)** → пункт меню **Комплекты (Kits)** → вкладка **CMake**.
- Если у вас есть записи в разделе **Auto-detected** — найдите среди них CMake с версией 3.26.3 и используйте его.
- Если нет — добавьте по кнопке "Добавить (Add)", файл может храниться в каталогах:
 - C:\Program Files (x86)\TensorCppDev-2022\CMake\bin\cmake.exe,
 - C:\Program Files\CMake\bin\cmake.exe,
 - C:\Program Files (x86)\CMake\bin\cmake.exe.
- Проверьте, что версия CMake определилась как 3.26.3, если нет — переустановите [tensor-cpp-dev-2.2](#).

Системная CMake в C:\Program Files (x86)\TensorCppDev-2022\CMake\bin\cmake.exe (По умолчанию) C:\Program Files (x86)\TensorCppDev-2022\CMake

те версию CMake

Системная CMake в C:\Program Files (x86)\TensorCppDev-2022\CMake\bin\cmake.exe

C:\Program Files (x86)\TensorCppDev-2022\CMake\bin\cmake.exe

Основной архитектурой приложений является Win32

Шаг 6.1: Настройка комплекта для приложений Win32

32-битные приложения в новом тулчейне собираются в режиме кросс-компиляции 64-битным компилятором.

1. Добавьте компилятор clang-cl — для C и C++.

- Перейдите в меню **Инструменты (Tools)** → **Параметры... (Options)** → пункт меню **Комплекты (Kits)** → вкладка **Компиляторы (Compilers)**.
- Нажмите кнопку "Добавить (Add)" и выберите clang-cl → C.
- в атрибуте **Инициализация (Initialization)** выберите C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvarsall.bat amd64_x86.
- в атрибуте **Путь к компилятору (Compiler path)** укажите путь до файла clang-cl, например, C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Tools\Llvm\x64\bin\clang-cl. Настройка clang-cl при добавлении для C++ выполняется аналогично.

Microsoft Visual C++ Compiler 17.4.33205.214 (amd64_x86)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (amd64)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (x86_amd64)	MSVC
Microsoft LLVM 64 bit based on MSVC2022	Clang
Microsoft LLVM 64 bit based on MSVC2022 (C, x86 64bit at C:\Qt\qtcreator-6.0.2\bin\clang\bin\clang.exe)	Clang
Microsoft Visual C++ Compiler 17.4.33205.214 (x86)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (amd64_x86)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (amd64)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (x86_amd64)	MSVC
Microsoft LLVM 64 bit based on MSVC2022	Clang

clang-cl Clang

clang-cl

Инициализация: C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvarsall.bat amd64_x86

Путь к компилятору: C:\Program Files (x86)\TensorCppDev-2022\Llvm\bin\clang-cl.exe

е путь к компилятору

- Нажмите кнопку "Применить (Apply)" для сохранения изменений.

2. Выполните создание комплекта (Kit).

sktop (x86-windows-msvc2022-pe-64bit)

ng

вой системе:

ва:

ва:

C: clang-cl

C++: clang-cl

PATH=C:\Python311-32;C:\Python311-32\Scripts;C:\Program Files (x86)\TensorCppDev-2022\ninja;\${PATH}

☐ Заставить компилятор MSVC выводить сообщения в UTF-8

Обнаруженный CDB в C:\Program Files (x86)\Windows Kits\10\Debuggers\x86\cdb.exe

Нет

- Перейдите в меню **Инструменты (Tools)** → **Параметры... (Options)** → пункт меню **Комплекты (Kits)** → вкладка **Kits (Комплекты)**.
- Нажмите кнопку "Добавить" для добавления нового комплекта.
- В атрибуте **Имя (Name)** установите произвольное имя комплекта.
Только для разработчиков мобильного приложения: для удобства использования рекомендуется выбрать имя, которое будет соответствовать архитектуре, под которую собирается проект мобильного приложения.
- В атрибуте **Компилятор (Compiler)** укажите:
 - для C — clang-cl.
 - для C++ — clang-cl.
- В атрибуте **Среда (Environment)** добавьте путь до Python 3.11 32bit, например:

```
1 PATH=C:\Python311-32;C:\Python311-32\Scripts;C:\Program Files
  (x86)\Microsoft Visual
  Studio\2022\BuildTools\Common7\IDE\CommonExtensions\Microsoft\CMake
  \Ninja;${PATH}
```

```
1 CMAKE_ASM_COMPILER_TARGET:STRING=i686-windows
2 CMAKE_CXX_COMPILER_TARGET:STRING=i686-windows
3 CMAKE_C_COMPILER_TARGET:STRING=i686-windows
4 CMAKE_SYSTEM_NAME:STRING=windows
5 CMAKE_SYSTEM_PROCESSOR:STRING=i686
```

В итоге конфигурация CMake должна выглядеть так:

```
1 CMAKE_ASM_COMPILER_TARGET:STRING=i686-windows
2 CMAKE_CXX_COMPILER:STRING=%{Compiler:Executable:Cxx}
3 CMAKE_CXX_COMPILER_TARGET:STRING=i686-windows
4 CMAKE_C_COMPILER:STRING=%{Compiler:Executable:C}
5 CMAKE_C_COMPILER_TARGET:STRING=i686-windows
```

```

6 CMAKE_PREFIX_PATH:STRING=%{Qt:QT_INSTALL_PREFIX}
7 CMAKE_SYSTEM_NAME:STRING=windows
8 CMAKE_SYSTEM_PROCESSOR:STRING=i686
9 QT_QMAKE_EXECUTABLE:STRING=%{Qt:qmakeExecutable}

```

- Нажмите кнопку "Применить (Apply)" для сохранения изменений.

Шаг 6.2: Настройка комплекта для приложений Win64

Важно!

Для приложений Win64 платформа поддерживает только статическую конфигурацию. Установите дополнительно [Python 3.11 64 bit](#).

1. Добавьте компилятор clang-cl — для C и C++.

- Перейдите в меню **Инструменты (Tools)** → **Параметры... (Options)** → пункт меню **Комплекты (Kits)** → вкладка **Компиляторы (Compilers)**.
- Нажмите кнопку "Добавить (Add)" и выберите clang-cl → C.
- Если вы ранее добавляли компилятор для [Win32](#), в атрибуте **Имя (Name)** укажите имя clang-cl x64, чтобы избежать путаницы.
- в атрибуте **Инициализация (Initialization)** выберите C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvarsall.bat amd64.
- в атрибуте **Путь к компилятору (Compiler path)** укажите путь до файла clang-cl, например, C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Tools\Llvm\x64\bin\clang-cl. Настройка clang-cl при добавлении для C++ выполняется аналогично.

Microsoft Visual C++ Compiler 17.4.33205.214 (amd64_x86)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (amd64)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (x86_amd64)	MSVC
Microsoft LLVM 64 bit based on MSVC2022	Clang
Microsoft LLVM 64 bit based on MSVC2022	Clang
Microsoft Visual C++ Compiler 17.4.33205.214 (x86)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (amd64_x86)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (amd64)	MSVC
Microsoft Visual C++ Compiler 17.4.33205.214 (x86_amd64)	MSVC
Microsoft LLVM 64 bit based on MSVC2022	Clang

Имя (Name)	Инициализация (Initialization)	Путь к компилятору (Compiler path)
clang-cl	Clang	

Имя (Name):	C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvarsall.bat amd64_x86
Путь к компилятору (Compiler path):	C:\Program Files (x86)\TensorCppDev-2022\Llvm\bin\clang-cl.exe

е путь к компилятору

- Нажмите кнопку "Применить (Apply)" для сохранения изменений.

2. Выполните создание комплекта (Kit).

ng x64
 sktop (x86-windows-msvc2022-pe-32bit) (по умолчанию)
 sktop (x86-windows-msvc2022-pe-64bit)

вой системе:

ва:

за:

Clang x64

Desktop

Локальный ПК (по умолчанию для Desktop)

Локальный ПК (по умолчанию для Desktop)

C: clang-cl x64

C++: clang-cl x64

PATH=C:\Program Files\Python311-64;C:\Program Files\Python311-64\Scripts;\${PATH}

☐ Заставить компилятор MSVC выводить сообщения в UTF-8

Обнаруженный CDB в C:\Program Files\Windows Kits\10\Debuggers\x64\cdb.exe

Нет

- Перейдите в меню **Инструменты (Tools) → Параметры... (Options) → пункт меню Комплекты (Kits) → вкладка Kits (Комплекты)**.
- Нажмите кнопку "Добавить" для добавления нового комплекта.
- Установите произвольное имя комплекта.

Только для разработчиков мобильного приложения: для удобства использования рекомендуется выбрать имя, которое будет соответствовать архитектуре, под которую собирается проект мобильного приложения.

- В атрибуте **Компилятор (Compiler)** укажите:
 - для C — clang-cl x64.
 - для C++ — clang-cl x64 .
- В атрибуте **Среда (Environment)** добавьте путь до Python 3.11 64 bit, например:

```
1 PATH=C:\Program Files\Python311-64;C:\Program Files\Python311-64\Scripts;${PATH}
```

- В атрибуте **Отладчик (Debugger)** укажите "CDB".
- В атрибуте **Программа CMake (CMake Tool)** укажите ранее настроенный CMake 3.26.3.
- В атрибуте **Генератор CMake (CMake Generator)** укажите "Ninja", дополнительный генератор - "CodeBlocks".
- В атрибуте **Конфигурация CMake (CMake Configuration)** добавьте следующие переменные:

```
1 CMAKE_CXX_COMPILER:STRING=%{Compiler:Executable:Cxx}
2 CMAKE_C_COMPILER:STRING=%{Compiler:Executable:C}
3 CMAKE_PREFIX_PATH:STRING=%{Qt:QT_INSTALL_PREFIX}
4 QT_QMAKE_EXECUTABLE:STRING=%{Qt:qmakeExecutable}
5 SBIS_ARCHITECTURE:STRING=x86_64
```

При статической сборке дополнительно добавьте переменные:

```
1 SBIS_STATIC_BUILD:BOOL=ON
2 SBIS_STATIC_RUNTIME:BOOL=ON
```

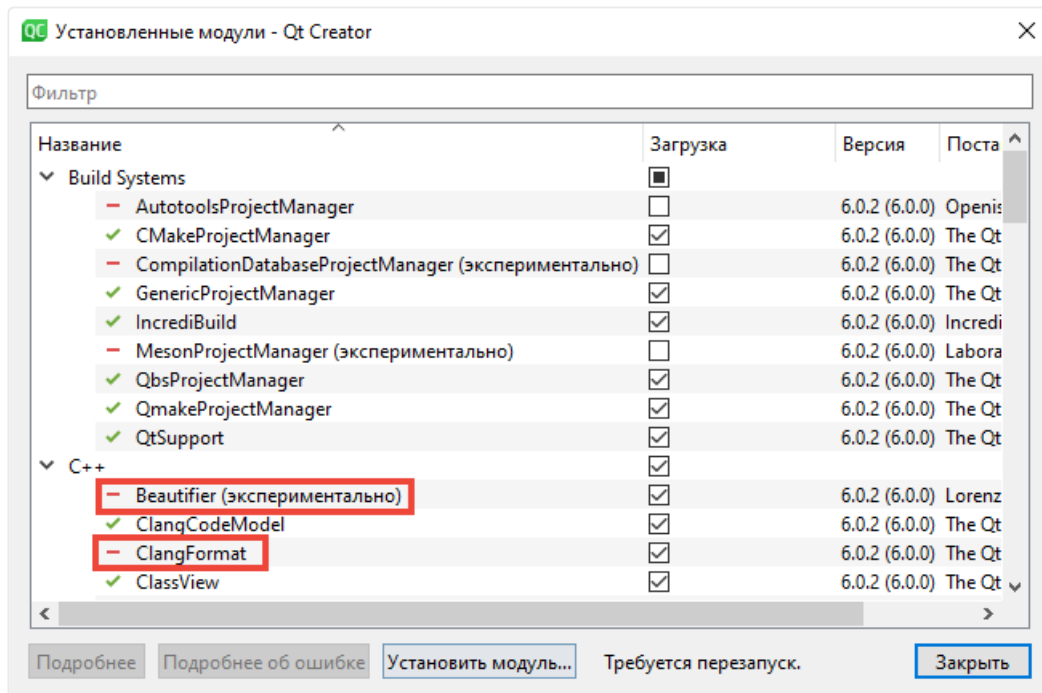
- Нажмите кнопку "Применить (Apply)" для сохранения изменений.

Шаг 7: Настройка стилизации кода в IDE Qt Creator (опционально)

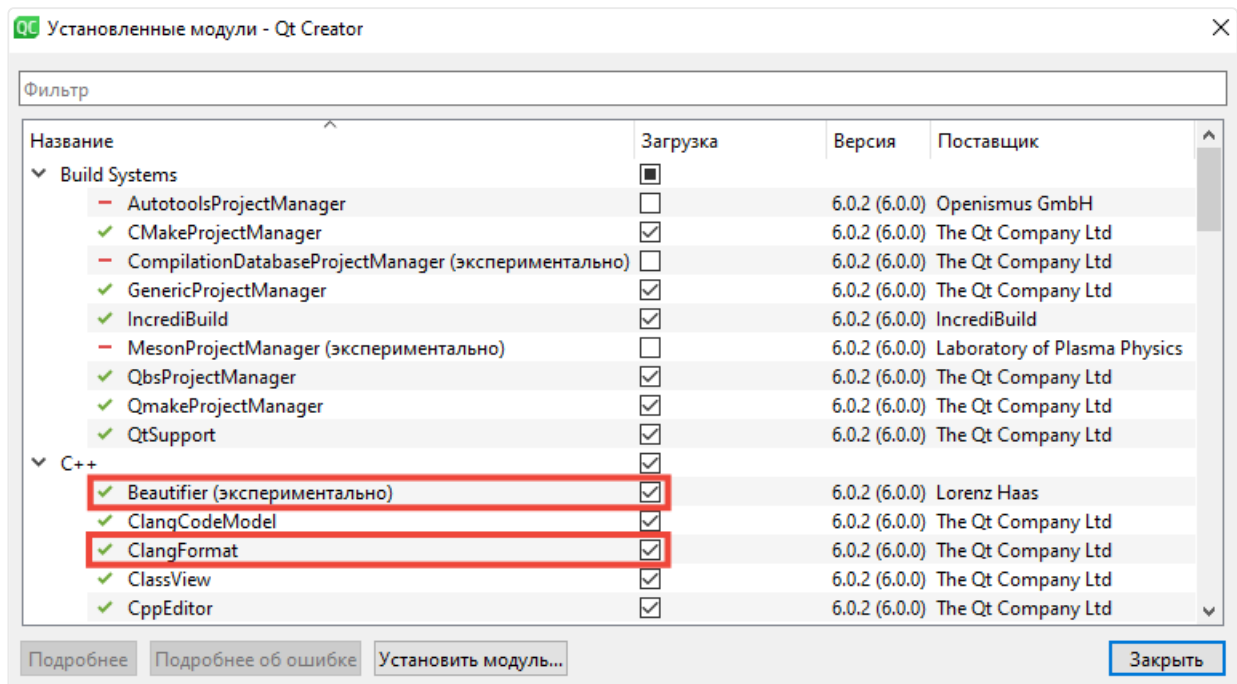
Для установки зависимостей загрузите с официального сайта [64-битную версию LLVM](#). В состав LLVM входит инструмент Clang Format, который используется для автоматического форматирования кода на C++.

Для настройки стилизации кода в Qt Creator выполните следующие шаги:

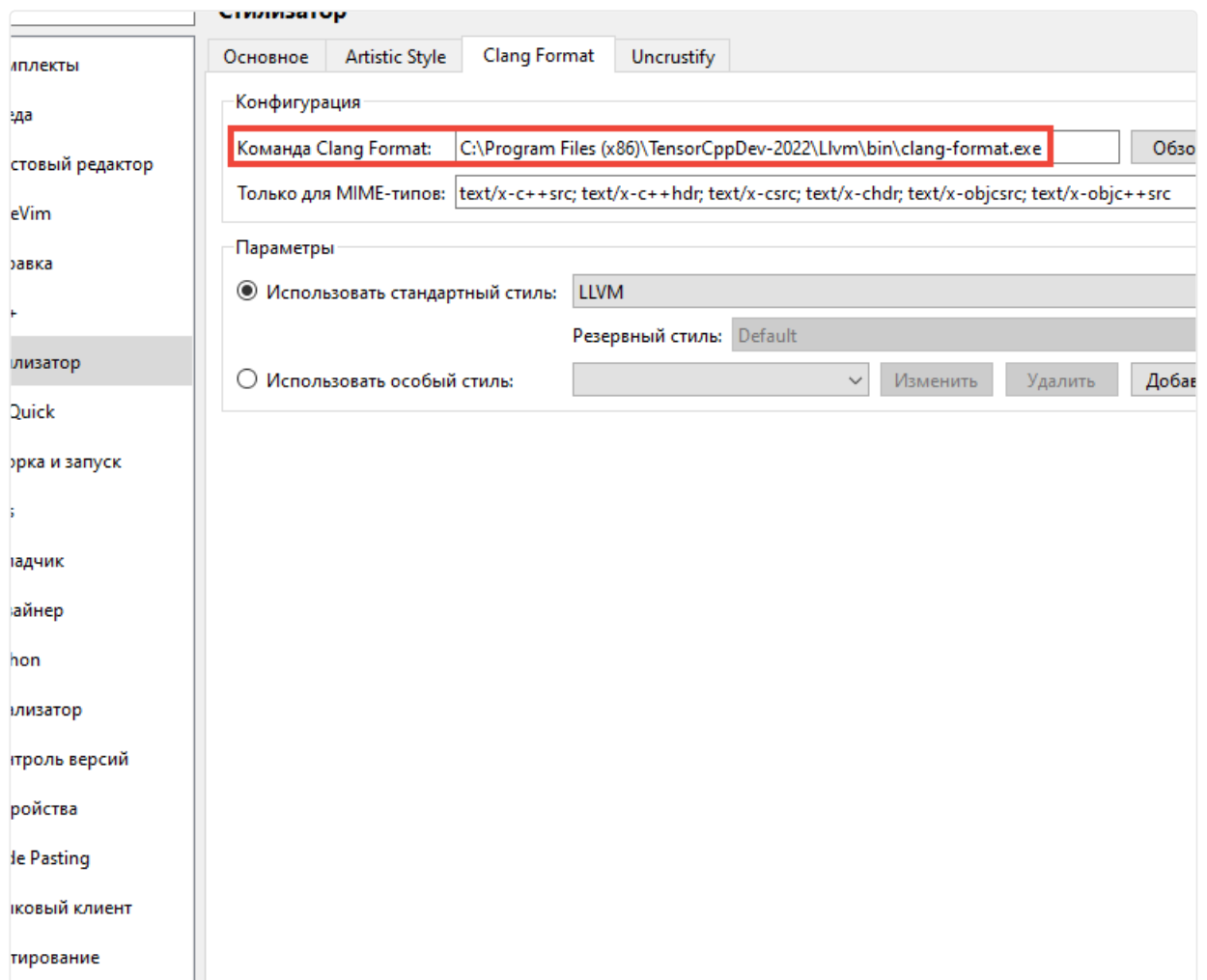
1. Запустите Qt Creator.
2. Перейдите в меню **Справка (Help) → О модулях (About Plugins)**.
3. Отметьте в списке модулей флаги у плагина Beautifier и инструмента ClangFormat. Нажмите кнопку "Закреть (Close)".



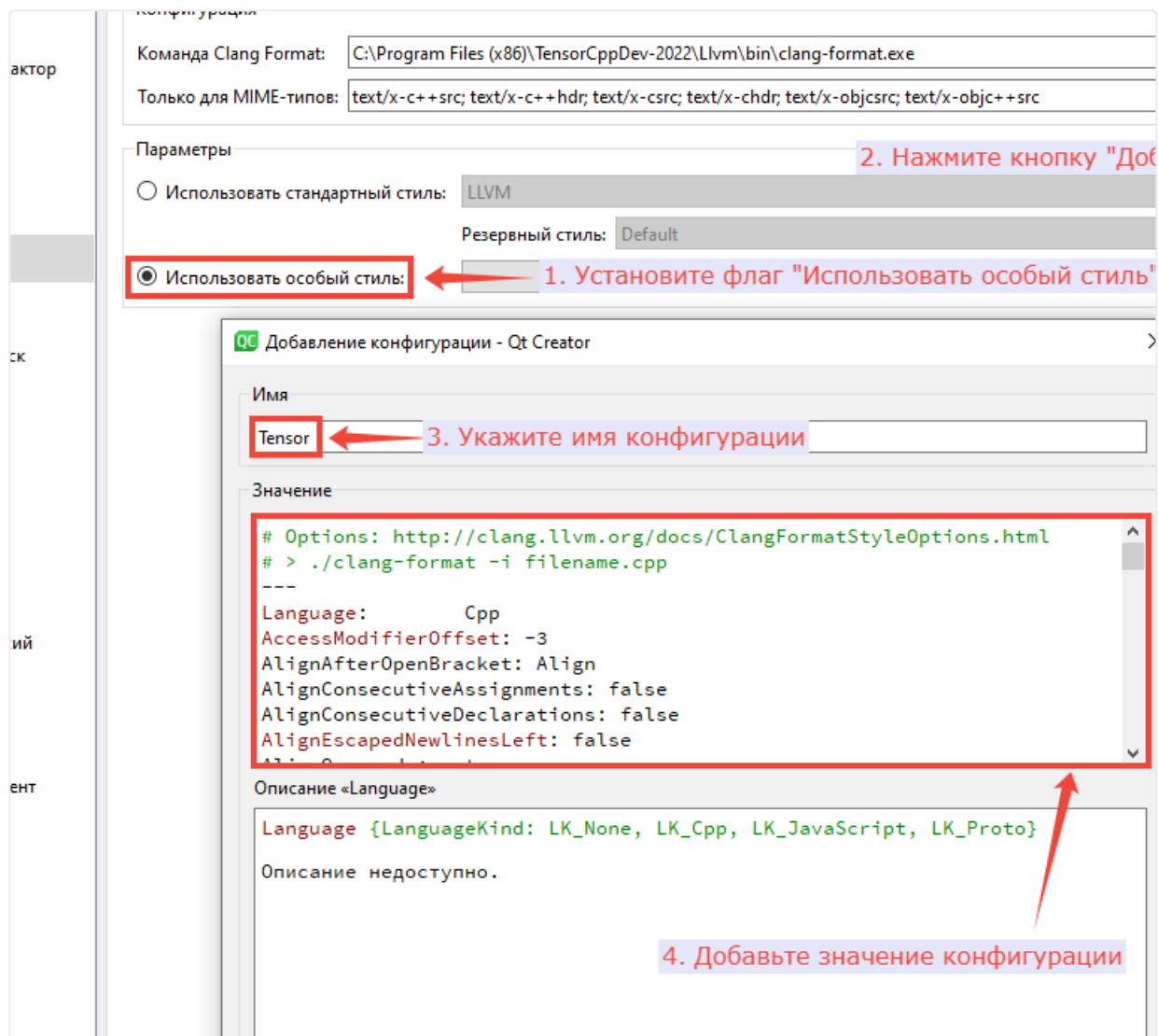
4. Перезапустите Qt Creator и проверьте в том же меню, что плагины установлены.



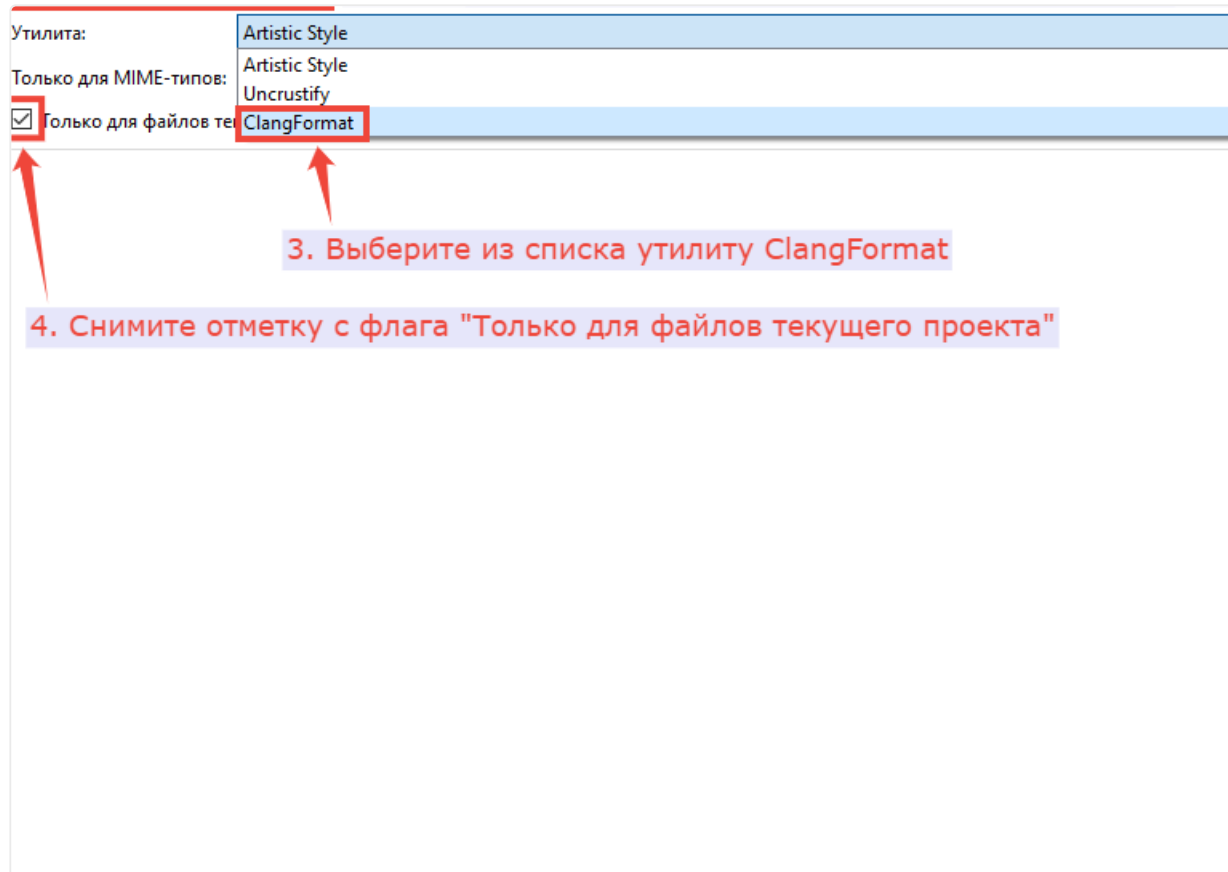
5. Перейдите в **Инструменты (Tools) → Параметры... (Options) → Стилизатор (Beautifier) → вкладка Clang Format**. В поле **Команда Clang Format** укажите путь к файлу clang-format.



- Отметьте флаг **Использовать особый стиль (Use customized style)** и добавьте конфигурацию с названием Tensor. В поле **Значение (Value)** вставьте настройки стиля из репозитория [sbis3](#) и нажмите "OK".



7. Перейдите на вкладку **Основное (Main)** для настройки автоформатирования.



Для выхода из окна **Параметры... (Options)** нажмите кнопку "OK".

Для проверки работы стилизатора добавьте переносы или табуляции в исходный код файла С++ и нажмите "Сохранить (Save)" (Ctrl + S).